

Comparing Smooth Basis Types

GAM.jl Contributors

Table of contents

Overview	1
Setup	2
Simulating data	2
Fitting models with different bases	3
Comparing EDF and deviance	4
Comparing smooth estimates	4
Visualizing basis functions	6
When to use which basis	7
Summary	8

Overview

GAMs represent smooth functions as linear combinations of **basis functions**. The choice of basis affects the shape of the fitted smooth, computational cost, and numerical properties. GAM.jl supports several basis types:

Symbol	Basis	Description
:tp	Thin plate regression spline	Default. Optimal in a certain sense; no knot placement needed
:cr	Cubic regression spline	Cubic spline with knots at data quantiles; efficient for 1D
:ps	P-spline	B-spline basis with difference penalty
:gp	Gaussian process	Squared-exponential covariance as a basis

This vignette fits the same data with each basis and compares the results.

Setup

```
using GAM
using StatsAPI: deviance, r2

using DataFrames
using Random
using Plots
```

Simulating data

We simulate $n = 300$ observations from a function with both broad and fine-scale structure:

$$y_i = \sin(2\pi x_i) + 0.5 \sin(6\pi x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, 0.5^2)$$

```
Random.seed!(42)
n = 300
x = sort(rand(n))
f_true = sin.(2 .* x) .+ 0.5 .* sin.(6 .* x)
y = f_true .+ 0.5 .* randn(n)
df = DataFrame(x = x, y = y)
```

	x	y
	Float64	Float64
1	0.000715255	0.654687
2	0.00119567	0.0229512
3	0.0100473	0.0491133
4	0.016095	0.540544
5	0.0269848	-0.0270783
6	0.0301856	0.202606
7	0.0369673	1.16376
8	0.0379955	0.538741
9	0.040211	0.76023
10	0.0424826	0.803596
11	0.0467336	-0.0516383
12	0.0467959	0.147176
13	0.0484837	1.15456
14	0.0513185	0.381877
15	0.0552982	0.404848
16	0.055968	1.10396
17	0.0588083	0.783381
18	0.0589488	1.7587
19	0.0604437	0.737216
20	0.0651857	1.35085
21	0.066672	1.58092
22	0.0726606	-0.0850994
23	0.0786647	1.31219
24	0.0793339	1.9222
...	...	...

Fitting models with different bases

We fit the same formula with each basis type, using $k = 20$ basis functions:

```

bases = [:tp, :cr, :ps, :gp]
models = Dict{Symbol, GamModel}()

for bs in bases
    models[bs] = gam(GamFormula(:y, Symbol[], true, [s(:x, k=20, bs=bs)]), df)
end

```

Comparing EDF and deviance

```
println("Basis    EDF        Deviance    Dev.Expl(%)")
println(" " ^ 50)
for bs in bases
    m = models[bs]
    e = round(edf(m)[1]; digits = 2)
    d = round(deviance(m); digits = 2)
    de = round(r2(m) * 100; digits = 1)
    println("$ (rpad(bs, 8)) $(lpad(string(e), 8))  $(lpad(string(d), 10))  $(lpad(string(de), 10))")
end
```

Basis	EDF	Deviance	Dev.Expl(%)
tp	11.31	72.54	70.6
cr	11.39	72.51	70.6
ps	11.01	73.05	70.4
gp	13.47	71.49	71.0

Comparing smooth estimates

We evaluate each smooth on the same grid and compare:

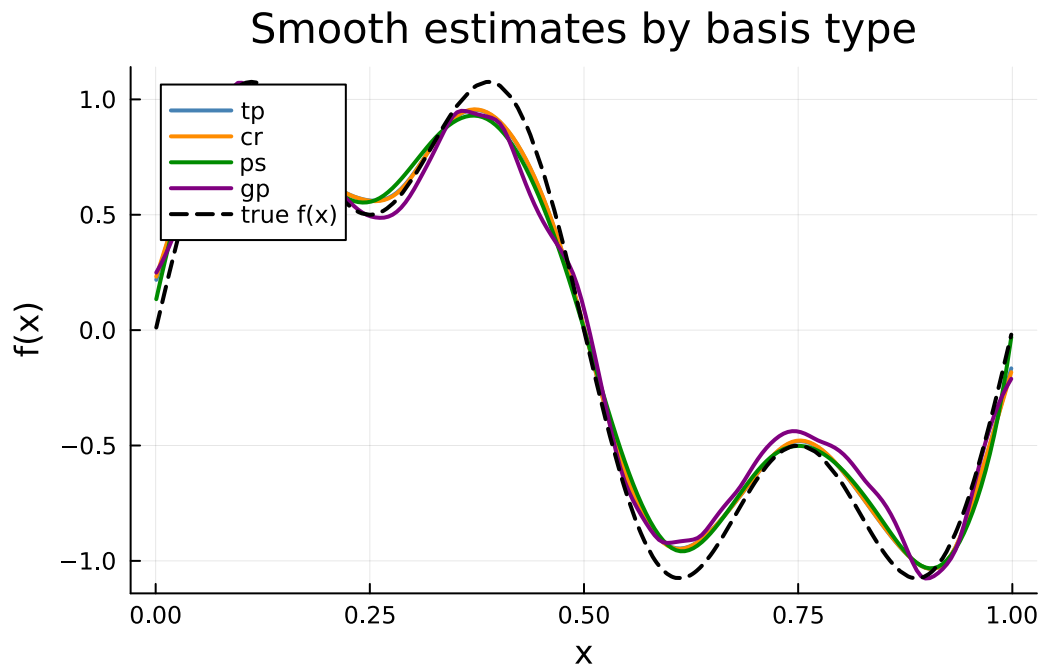
```
p = plot(xlabel = "x", ylabel = "f(x)",
         title = "Smooth estimates by basis type", legend = :topleft)

colors = [:steelblue, :darkorange, :green4, :purple]

for (i, bs) in enumerate(bases)
    se = smooth_estimates(models[bs]; n = 200)
    plot!(p, se.covariates[:x], se.estimate;
          label = string(bs),
          linewidth = 2,
          color = colors[i])
end

plot!(p, x, f_true;
      label = "true f(x)",
      linestyle = :dash,
      linewidth = 2,
```

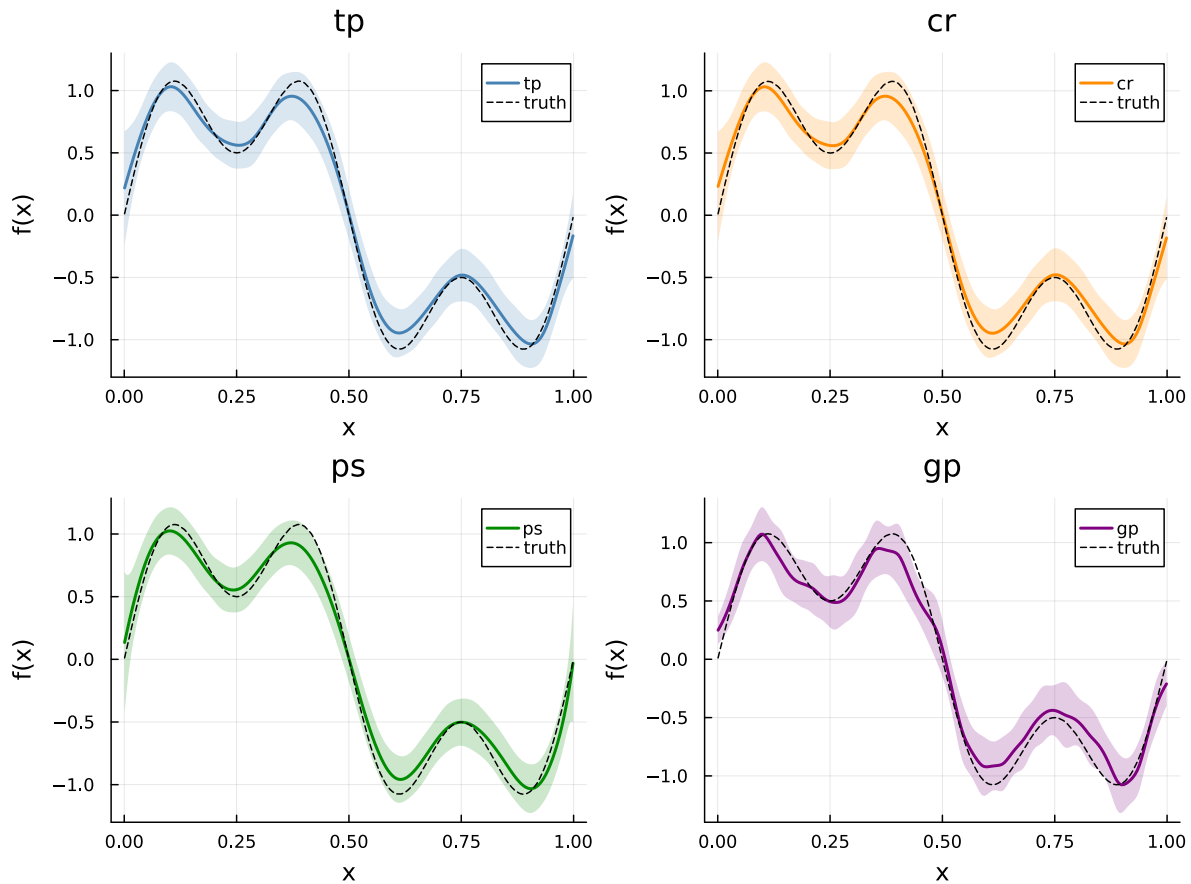
```
color = :black)
p
```



We can also show each basis with its confidence band:

```
plots = []
for (i, bs) in enumerate(bases)
    se = smooth_estimates(models[bs]; n = 200)
    pi = plot(se.covariates[:x], se.estimate;
        ribbon = 2 .* se.se,
        fillalpha = 0.2,
        label = string(bs),
        linewidth = 2,
        color = colors[i],
        title = string(bs),
        xlabel = "x",
        ylabel = "f(x)")
    plot!(pi, x, f_true;
        label = "truth",
        linestyle = :dash,
        color = :black)
    push!(plots, pi)
```

```
end
plot(plots...; layout = (2, 2), size = (800, 600))
```



Visualizing basis functions

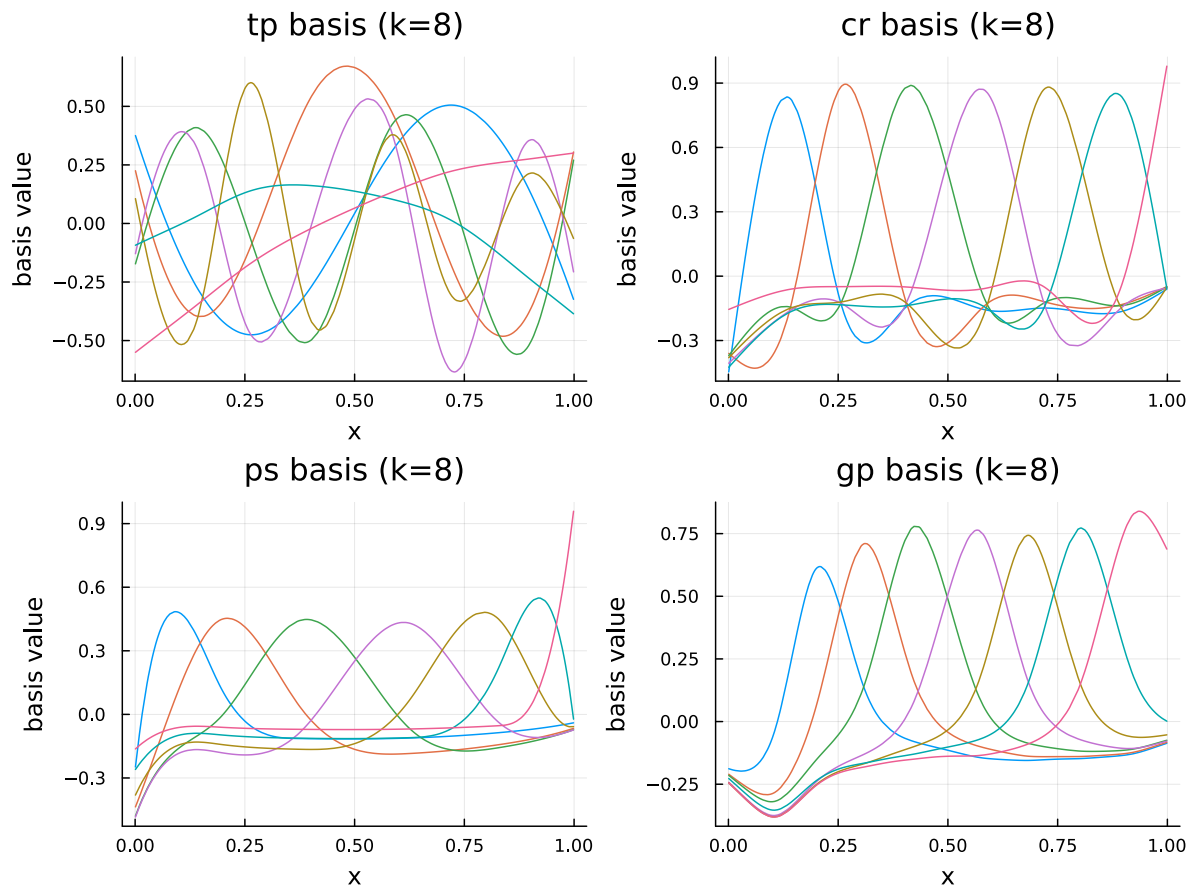
To understand how each basis works, we can examine the model matrix columns for a small number of basis functions ($k = 8$):

```
plots_basis = []
for (i, bs) in enumerate(bases)
  m_small = gam(GamFormula(:y, Symbol[], true, [s(:x, k=8, bs=bs)]), df)
  X_smooth = m_small.smooths[1].X
  k_cols = size(X_smooth, 2)
  pi = plot(title = "$ (bs) basis (k=8)", xlabel = "x", ylabel = "basis value",
    legend = false)
```

```

for j in 1:k_cols
    order = sortperm(df.x)
    plot!(pi, df.x[order], X_smooth[order, j]; linewidth = 1)
end
push!(plots_basis, pi)
end
plot(plots_basis...; layout = (2, 2), size = (800, 600))

```



When to use which basis

- **Thin plate (:tp):** The default choice. Works well in any dimension. Optimal smoothness in a certain mathematical sense. Slightly more expensive than knot-based alternatives for large datasets.
- **Cubic regression spline (:cr):** Efficient for 1D smoothing with knots at data quantiles. Produces smooth curves that are natural cubic splines. Good default for univariate

smooths.

- **P-spline (:ps)**: B-spline basis with a difference penalty on adjacent coefficients. Evenly spaced knots. Computationally efficient and well-behaved, especially for evenly sampled data.
- **Gaussian process (:gp)**: Uses a squared-exponential covariance kernel. Produces very smooth curves. Useful when the underlying function is believed to be infinitely differentiable.

Summary

In this vignette we:

1. Simulated bumpy data with both low- and high-frequency components
2. Fitted GAMs using four different basis types (TP, CR, PS, GP)
3. Compared EDF, deviance, and smooth estimates across bases
4. Visualized the raw basis functions for each type
5. Discussed when each basis is most appropriate

The next vignette demonstrates models with multiple smooth terms.